

HW1 - Neural Networks

Google Colab [Optional]

If you are using Google Colab, configure and run the cell below to mount your Google Drive and set up the correct working directory for the assignment. You may ignore this cell if you are working locally.

```
In [ ]: from google.colab import drive
import os

# 1. Mounting Google Drive: This allows Colab to access files in your Google
drive.mount('/content/drive')

# 2. Tell Colab where to find your assignment files and where to save your w

# TODO: Enter the relative path in your Google Drive of the coding assignment
FOLDERNAME = "Deep Learning Homework/hw1" # e.g. 'cs7643/ps0/'

assert FOLDERNAME is not None, "[!] Enter the foldername."

FULL_PATH = "/content/drive/MyDrive/" + FOLDERNAME
assert os.path.exists(FULL_PATH), "Make sure your FOLDERNAME is correct"
```

```
In [ ]: %cd $FULL_PATH
```

```
In [ ]: !uv pip install -r requirements.txt --system
```

Download Data

Sometimes data may not be downloaded properly from Google Drive due to rate limits when running the following command. We provide backup dataset files on Canvas. Please download them from there if you encounter issues when downloading from Google Drive.

```
In [ ]: !ls "/content/drive/MyDrive/Deep Learning Homework/hw1"
```

```
In [ ]: !cd data && python ./get_data.py
```

Tests

```
In [ ]: # uncomment to run individual tests
# !python -m pytest "tests/test_loading.py" -v
# !python -m pytest "tests/test_activation.py" -v
```

```
# !python -m pytest "tests/test_loss.py" -v
# !python -m pytest "tests/test_network.py" -v
!python -m pytest "tests/test_training.py" -v
```

```
In [ ]: # uncomment to run all tests
!python -m pytest "tests" -v
```

Homework 1 Writeup

After you have implemented all TODOs and passed all tests, you are ready to train your models! In this section, run the cells below to experiment with different hyperparameters. There are no single correct answers here, we are just trying to get you to think about what you observe and explain why you think the results are the way they are.

- Name: Venkata Aashutosh Aripirala
- GT Email: av84@gatech.edu
- GT ID: 904173725

Baselines

```
In [12]: from utils import display_training_plots
```

Softmax Baseline Plot [1 pt]

```
In [13]: # train softmax model
!python train_model.py --config configs/config_softmax.yaml --run_name softmax
```

```
Starting training run: softmax_baseline
Configuration: {'batch_size': 64, 'learning_rate': 0.1, 'reg': 0.001, 'epochs': 10, 'momentum': 0.9, 'debug': True, 'type': 'SoftmaxRegression'}
Loading training data...
Training data loaded with 60000 images
Loading testing data...
Testing data loaded with 10000 images
Using SoftmaxRegression model
Using SGD optimizer with lr=0.1, reg=0.001

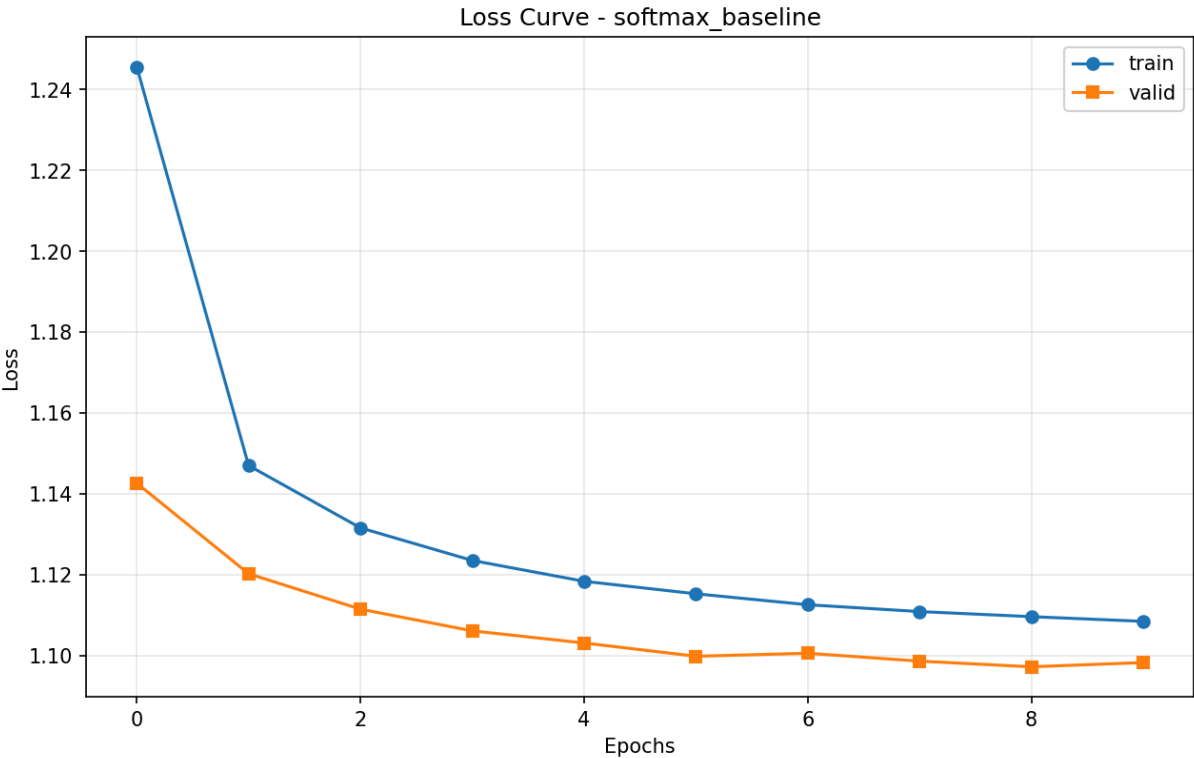
Starting training for 10 epochs...
Training: 100% 10/10 [00:32<00:00, 3.26s/it]

Training completed! Test Accuracy: 0.5778, Best Val Accuracy: 0.5792
Training log saved to: plots/softmax_baseline.log
Loss plot saved to: plots/softmax_baseline_loss.png
Accuracy plot saved to: plots/softmax_baseline_accuracy.png
```

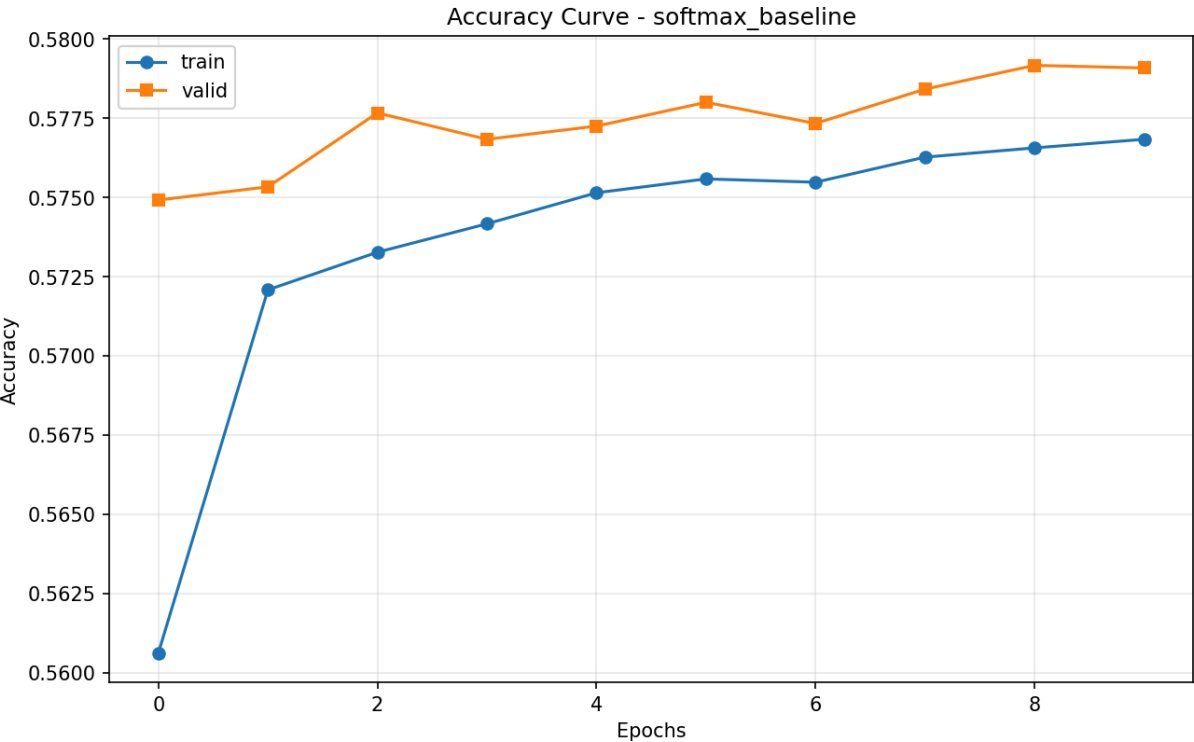
```
In [14]: display_training_plots("softmax_baseline")
```

Run: softmax_baseline
Test Accuracy: 0.5778
Best Validation Accuracy: 0.5792
Model: SoftmaxRegression, LR: 0.1, Reg: 0.001

Loss Curve:



Accuracy Curve:



Two-Layer Baseline Plot [1 pt]

```
In [15]: # train two layer neural network
!python train_model.py --config configs/config_twolayer.yaml --run_name twol
```

```
Starting training run: twolayer_baseline
Configuration: {'batch_size': 64, 'learning_rate': 0.1, 'reg': 0.001, 'epoch
s': 10, 'momentum': 0.9, 'debug': True, 'type': 'TwoLayerNet', 'hidden_siz
e': 128}
Loading training data...
Training data loaded with 60000 images
Loading testing data...
Testing data loaded with 10000 images
Using TwoLayerNet model with hidden_size=128
Using SGD optimizer with lr=0.1, reg=0.001

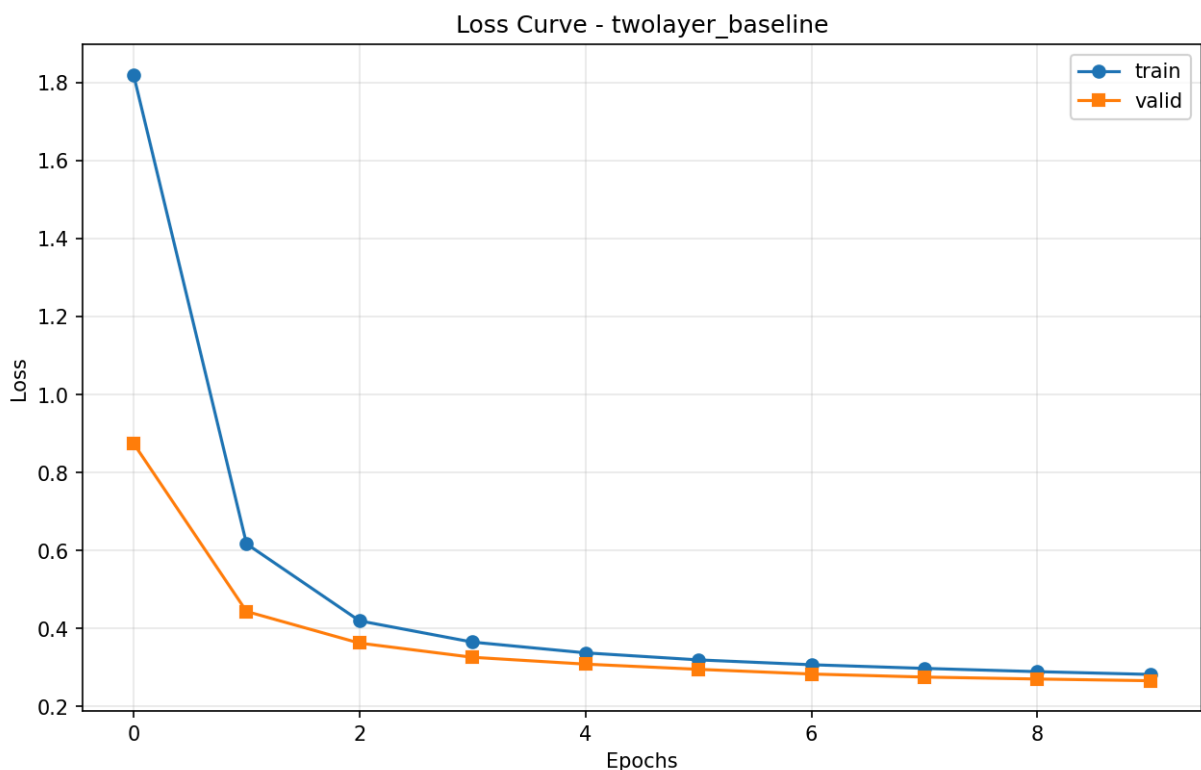
Starting training for 10 epochs...
Training: 100% 10/10 [01:04<00:00, 6.49s/it]

Training completed! Test Accuracy: 0.9253, Best Val Accuracy: 0.9271
Training log saved to: plots/twolayer_baseline.log
Loss plot saved to: plots/twolayer_baseline_loss.png
Accuracy plot saved to: plots/twolayer_baseline_accuracy.png
```

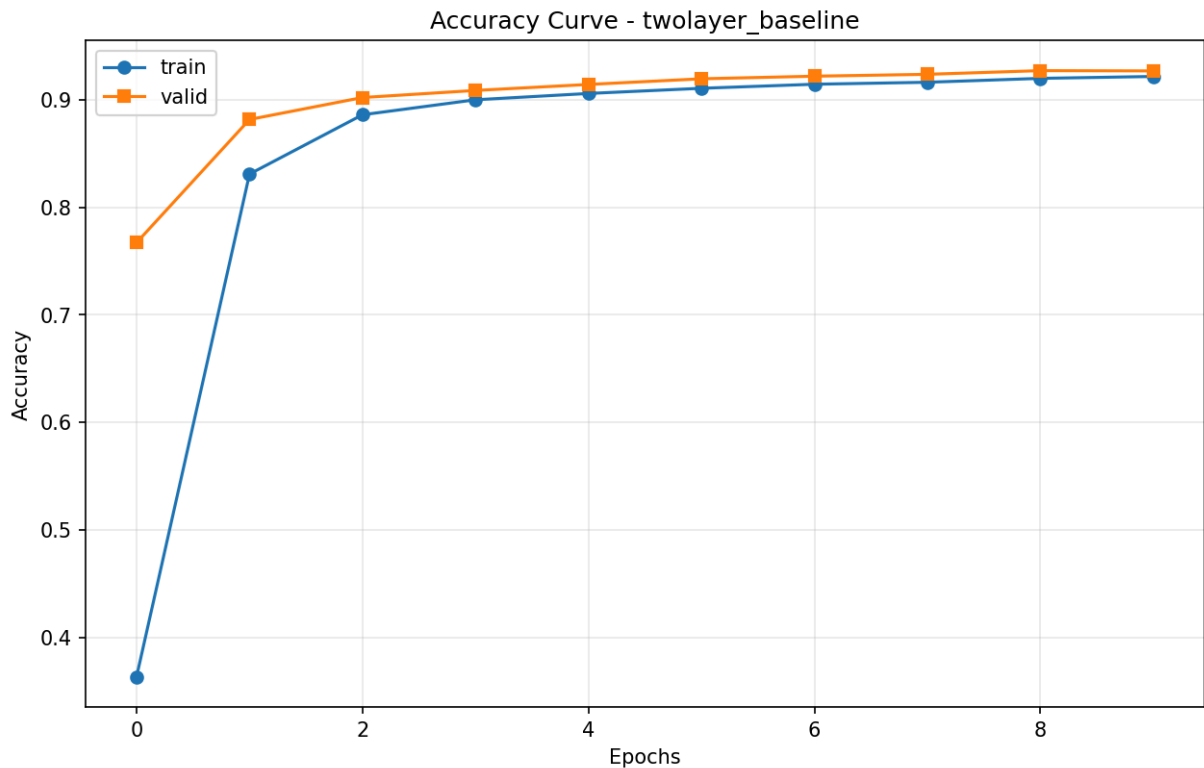
```
In [16]: display_training_plots("twolayer_baseline")
```

```
Run: twolayer_baseline
Test Accuracy: 0.9253
Best Validation Accuracy: 0.9271
Model: TwoLayerNet, LR: 0.1, Reg: 0.001
```

Loss Curve:



Accuracy Curve:



Hyperparameters

Learning Rates

- Train the Two Layer Neural Network with various learning rates (while keeping all other hyperparameters constant). Make sure you have not changed `configs/config_twolayer.yaml`
 - $lr = 1$
 - $lr = 1e-1$
 - $lr = 1e-2$
 - $lr = 5e-2$

Plots [1pt]

```
In [17]: !python train_model.py --config configs/config_twolayer.yaml --run_name two_
```

```
Starting training run: two_layer_lr_1_0
Configuration: {'batch_size': 64, 'learning_rate': 1.0, 'reg': 0.001, 'epochs': 10, 'momentum': 0.9, 'debug': True, 'type': 'TwoLayerNet', 'hidden_size': 128}
Loading training data...
Training data loaded with 60000 images
Loading testing data...
Testing data loaded with 10000 images
Using TwoLayerNet model with hidden_size=128
Using SGD optimizer with lr=1.0, reg=0.001
```

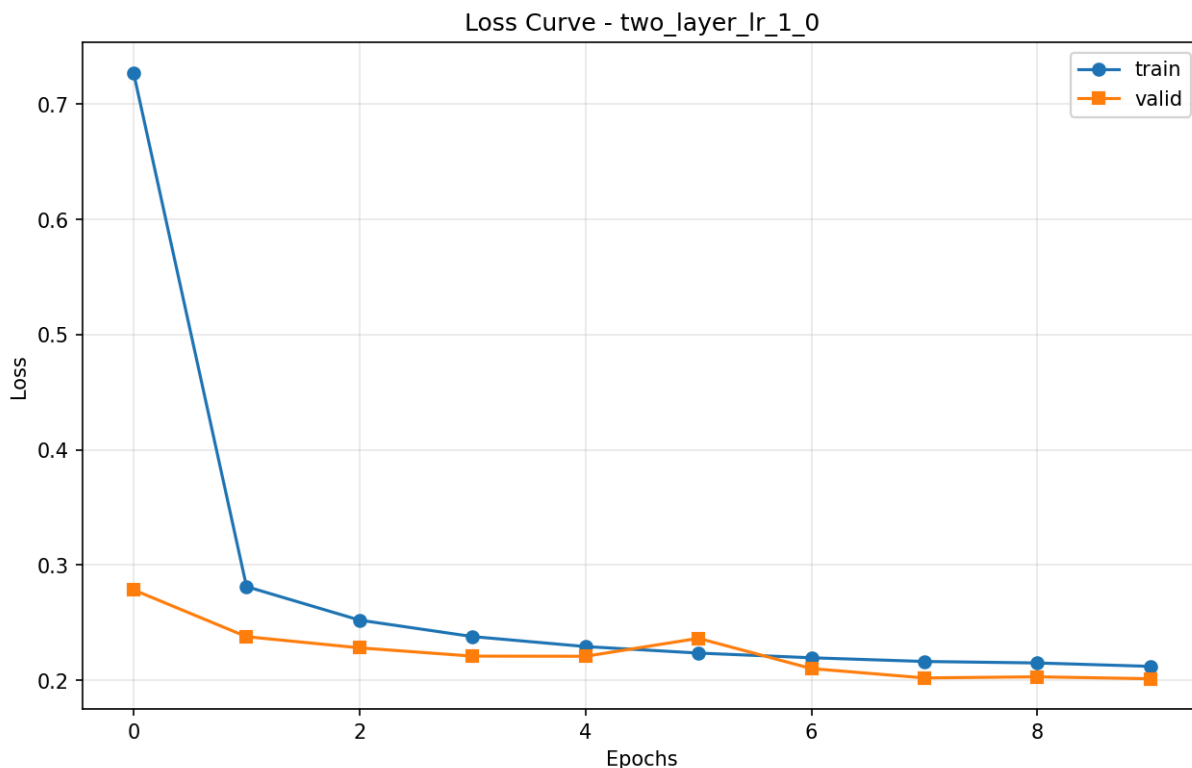
```
Starting training for 10 epochs...
Training: 100% 10/10 [01:07<00:00, 6.74s/it]
```

```
Training completed! Test Accuracy: 0.9452, Best Val Accuracy: 0.9480
Training log saved to: plots/two_layer_lr_1_0.log
Loss plot saved to: plots/two_layer_lr_1_0_loss.png
Accuracy plot saved to: plots/two_layer_lr_1_0_accuracy.png
```

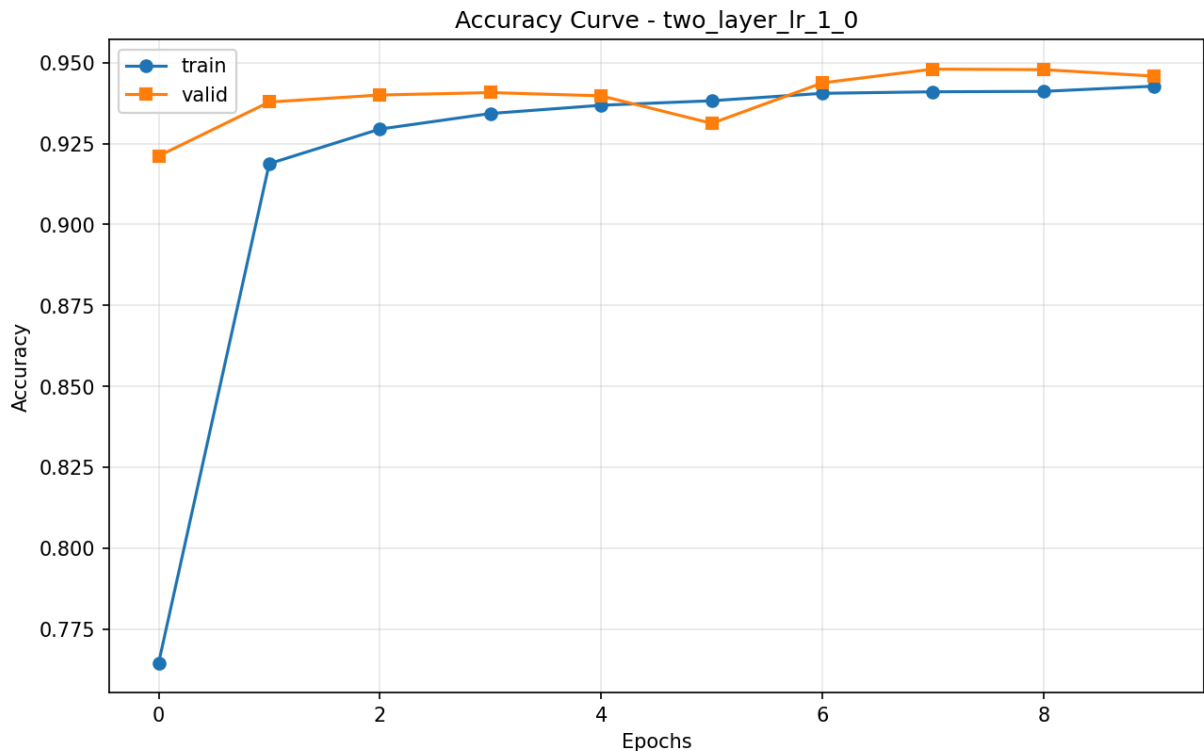
```
In [18]: display_training_plots("two_layer_lr_1_0")
```

```
Run: two_layer_lr_1_0
Test Accuracy: 0.9452
Best Validation Accuracy: 0.9480
Model: TwoLayerNet, LR: 1.0, Reg: 0.001
```

Loss Curve:



Accuracy Curve:



```
In [19]: !python train_model.py --config configs/config_twolayer.yaml --run_name two_
```

Starting training run: two_layer_lr_0_1

Configuration: {'batch_size': 64, 'learning_rate': 0.1, 'reg': 0.001, 'epoch s': 10, 'momentum': 0.9, 'debug': True, 'type': 'TwoLayerNet', 'hidden_siz e': 128}

Loading training data...

Training data loaded with 60000 images

Loading testing data...

Testing data loaded with 10000 images

Using TwoLayerNet model with hidden_size=128

Using SGD optimizer with lr=0.1, reg=0.001

Starting training for 10 epochs...

Training: 100% 10/10 [01:05<00:00, 6.51s/it]

Training completed! Test Accuracy: 0.9254, Best Val Accuracy: 0.9273

Training log saved to: plots/two_layer_lr_0_1.log

Loss plot saved to: plots/two_layer_lr_0_1_loss.png

Accuracy plot saved to: plots/two_layer_lr_0_1_accuracy.png

```
In [20]: display_training_plots("two_layer_lr_0_1")
```

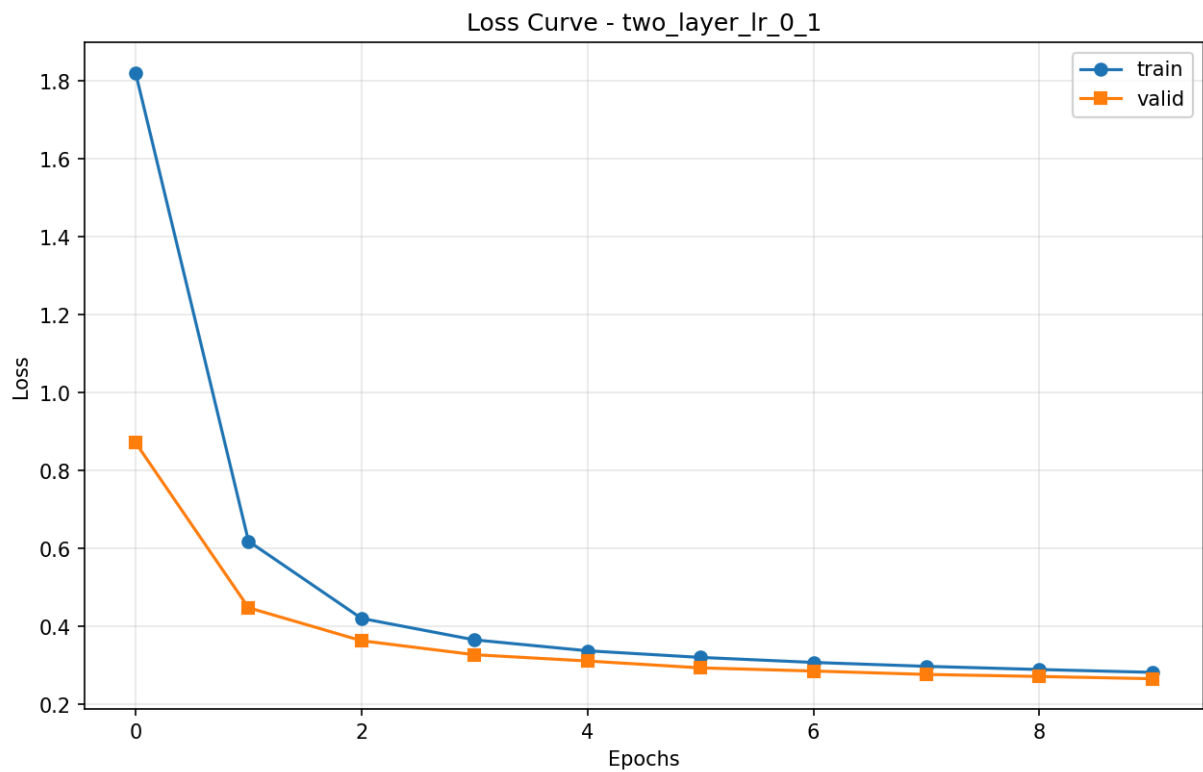
Run: two_layer_lr_0_1

Test Accuracy: 0.9254

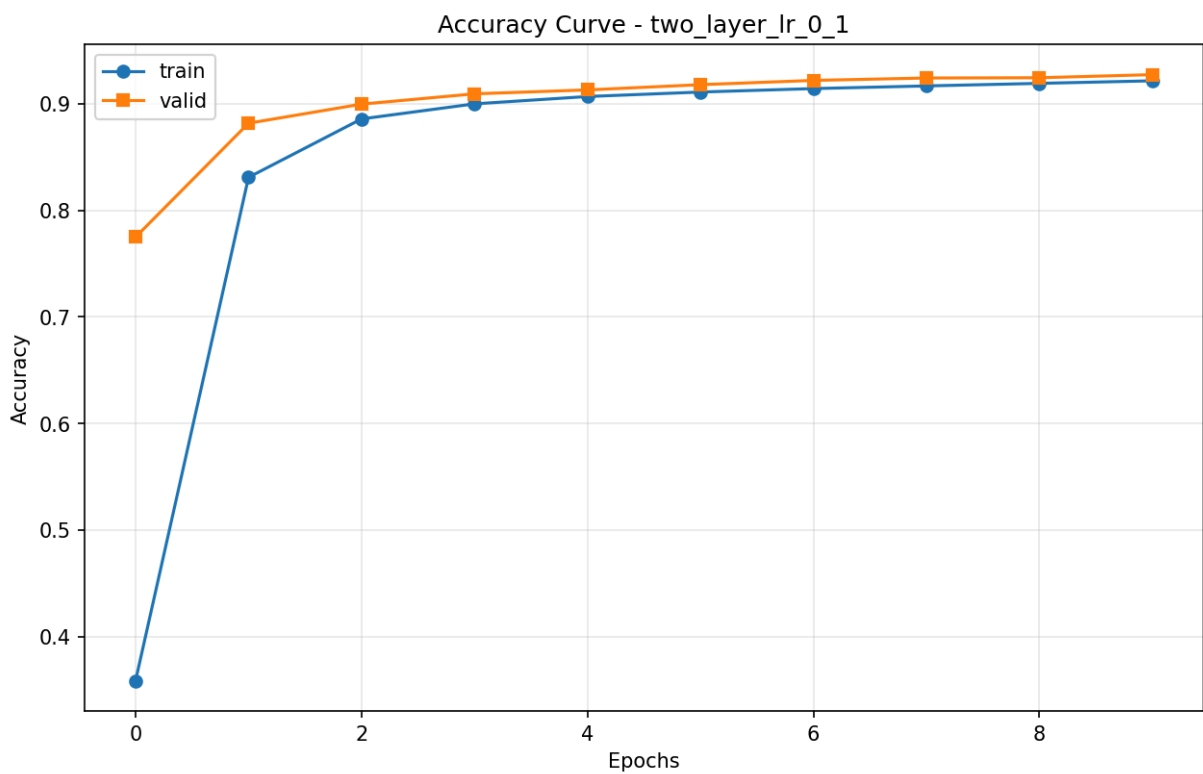
Best Validation Accuracy: 0.9273

Model: TwoLayerNet, LR: 0.1, Reg: 0.001

Loss Curve:



Accuracy Curve:



```
In [21]: !python train_model.py --config configs/config_twolayer.yaml --run_name two_
```



```
Starting training run: two_layer_lr_0_01
Configuration: {'batch_size': 64, 'learning_rate': 0.01, 'reg': 0.001, 'epochs': 10, 'momentum': 0.9, 'debug': True, 'type': 'TwoLayerNet', 'hidden_size': 128}
Loading training data...
Training data loaded with 60000 images
Loading testing data...
Testing data loaded with 10000 images
Using TwoLayerNet model with hidden_size=128
Using SGD optimizer with lr=0.01, reg=0.001
```

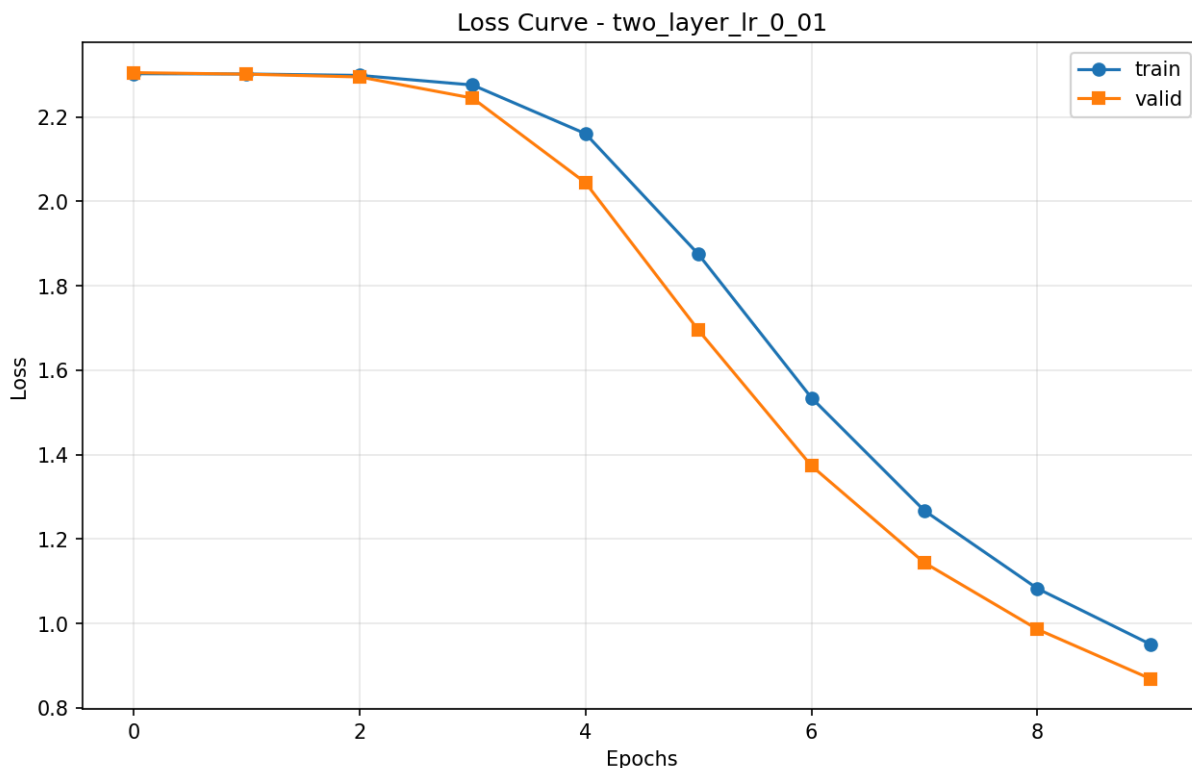
```
Starting training for 10 epochs...
Training: 100% 10/10 [01:04<00:00, 6.46s/it]
```

```
Training completed! Test Accuracy: 0.7614, Best Val Accuracy: 0.7701
Training log saved to: plots/two_layer_lr_0_01.log
Loss plot saved to: plots/two_layer_lr_0_01_loss.png
Accuracy plot saved to: plots/two_layer_lr_0_01_accuracy.png
```

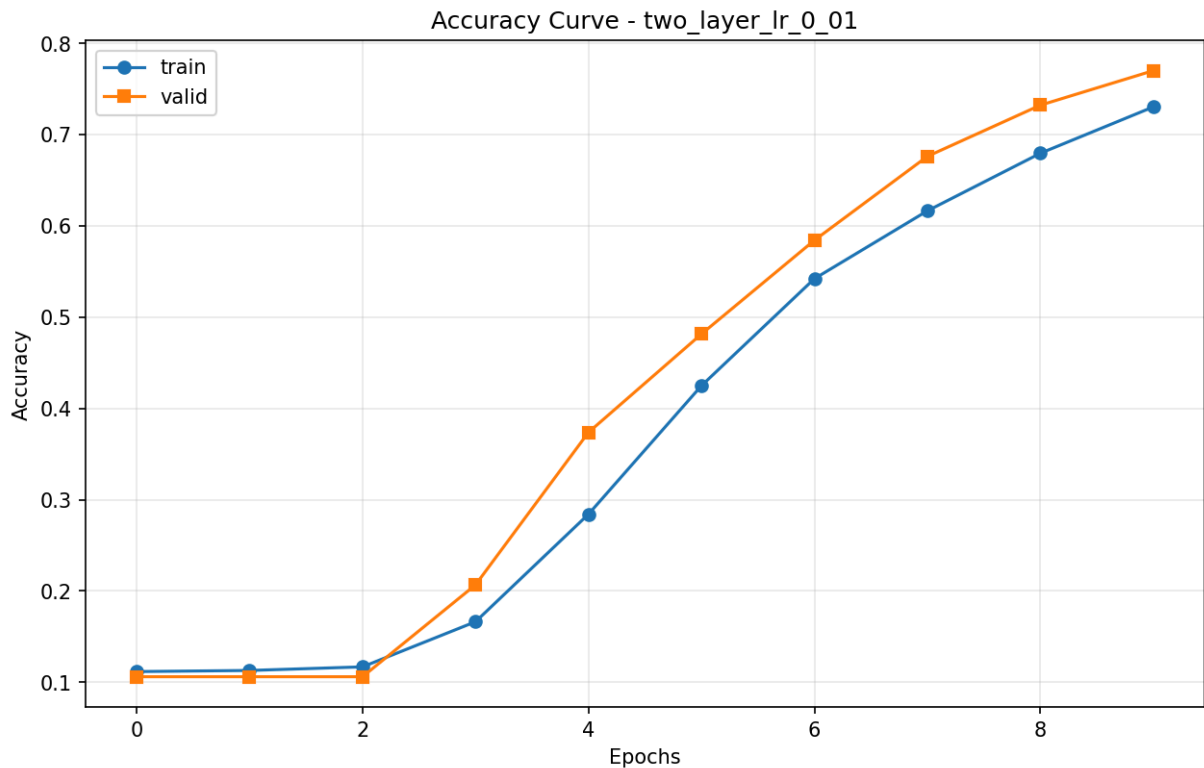
```
In [22]: display_training_plots("two_layer_lr_0_01")
```

```
Run: two_layer_lr_0_01
Test Accuracy: 0.7614
Best Validation Accuracy: 0.7701
Model: TwoLayerNet, LR: 0.01, Reg: 0.001
```

Loss Curve:



Accuracy Curve:



```
In [23]: !python train_model.py --config configs/config_twolayer.yaml --run_name two_
```

```
Starting training run: two_layer_lr_0_05
Configuration: {'batch_size': 64, 'learning_rate': 0.05, 'reg': 0.001, 'epochs': 10, 'momentum': 0.9, 'debug': True, 'type': 'TwoLayerNet', 'hidden_size': 128}
```

```
Loading training data...
```

```
Training data loaded with 60000 images
```

```
Loading testing data...
```

```
Testing data loaded with 10000 images
```

```
Using TwoLayerNet model with hidden_size=128
```

```
Using SGD optimizer with lr=0.05, reg=0.001
```

```
Starting training for 10 epochs...
```

```
Training: 100% 10/10 [01:08<00:00, 6.82s/it]
```

```
Training completed! Test Accuracy: 0.9134, Best Val Accuracy: 0.9163
```

```
Training log saved to: plots/two_layer_lr_0_05.log
```

```
Loss plot saved to: plots/two_layer_lr_0_05_loss.png
```

```
Accuracy plot saved to: plots/two_layer_lr_0_05_accuracy.png
```

```
In [24]: display_training_plots("two_layer_lr_0_05")
```

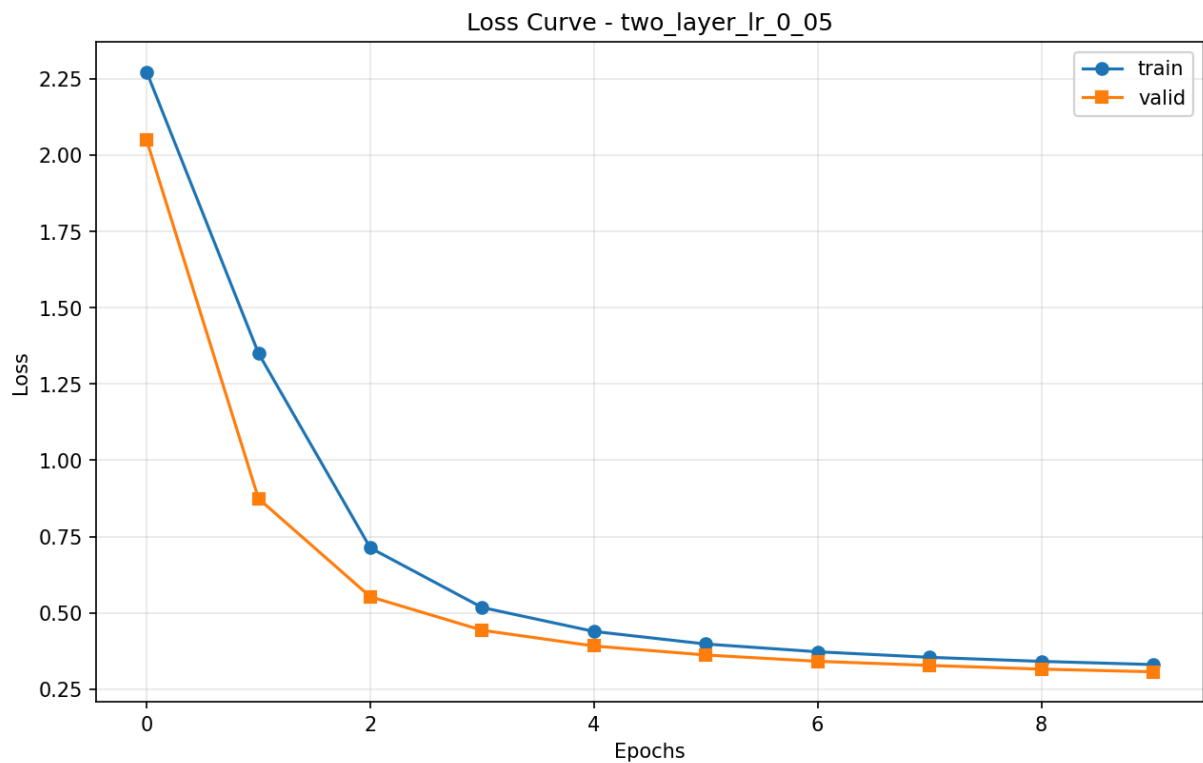
```
Run: two_layer_lr_0_05
```

```
Test Accuracy: 0.9134
```

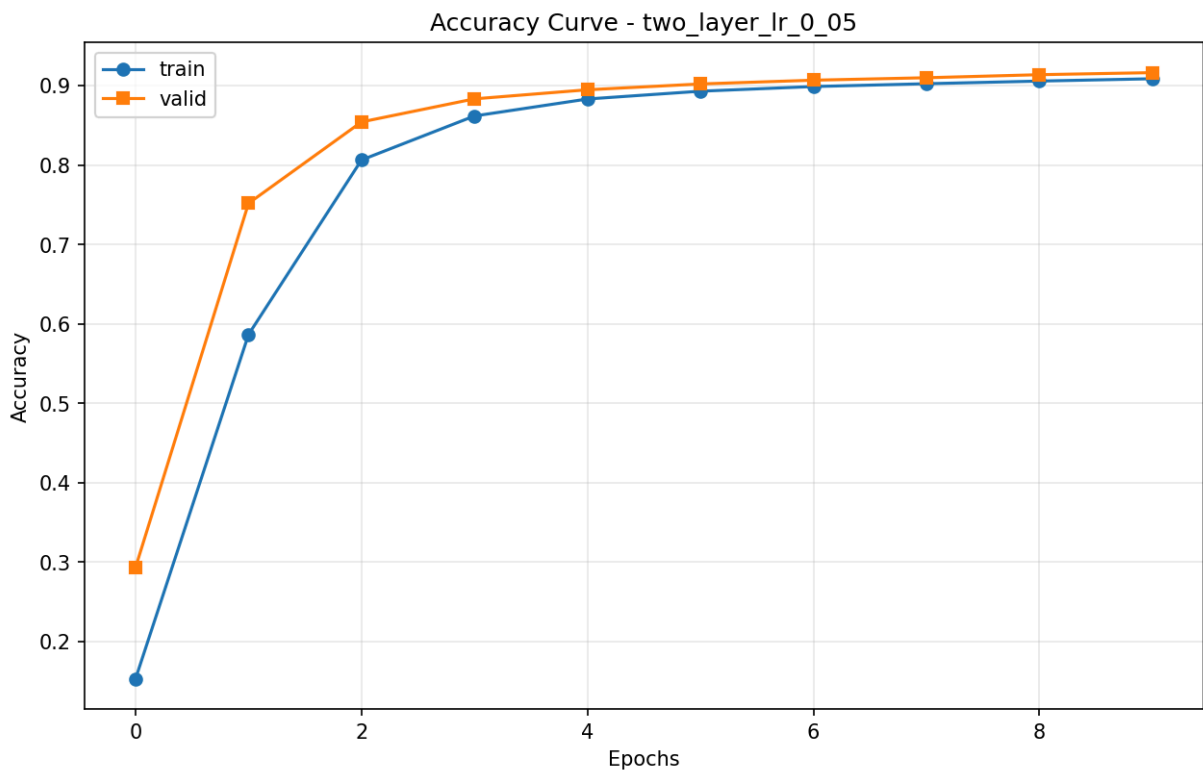
```
Best Validation Accuracy: 0.9163
```

```
Model: TwoLayerNet, LR: 0.05, Reg: 0.001
```

```
-----
Loss Curve:
```



Accuracy Curve:



TODO: Observations [1 pt]

What differences do you observe between different learning rates? Why do you think this is the case?

Since we're only training for 10 epochs, the smaller learning rates (particularly 0.01) doesn't have enough time to converge, clearly performing the worst among all learning rates. As we go up in learning rate magnitude, the improvement is very evident. The reason in a case like this is, we don't need a small learning rate because the dataset itself isn't the most difficult to learn, so we didn't even need to train much to get a really good accuracy score on the high-learning rate experiments.

Having said that, we can clearly also observe that the curves for the smaller learning rates are much smoother, with monotonically decreasing loss, and increasing accuracy. While the $lr=1.0$ achieves the best loss and accuracy, the optimizer takes substantial steps per batch, escaping flat regions quickly and converging rapidly toward a good local minimum, though higher rates introduce minor oscillations as the model approaches flatter regions of the objective surface.

Regularization

- Train the Two Layer Neural Network with various regularization coefficients (while keeping all other hyperparameters constant). Make sure you have not changed `configs/config_twolayer.yaml`
 - `reg = 1e-1`
 - `reg = 1e-2`
 - `reg = 1e-3`
 - `reg = 1e-4`
 - `reg = 1`

Plots [1 pt]

```
In [25]: !python train_model.py --config configs/config_twolayer.yaml --run_name two_
```

```
Starting training run: two_layer_reg_0_1
Configuration: {'batch_size': 64, 'learning_rate': 0.1, 'reg': 0.1, 'epochs': 10, 'momentum': 0.9, 'debug': True, 'type': 'TwoLayerNet', 'hidden_size': 128}
Loading training data...
Training data loaded with 60000 images
Loading testing data...
Testing data loaded with 10000 images
Using TwoLayerNet model with hidden_size=128
Using SGD optimizer with lr=0.1, reg=0.1
```

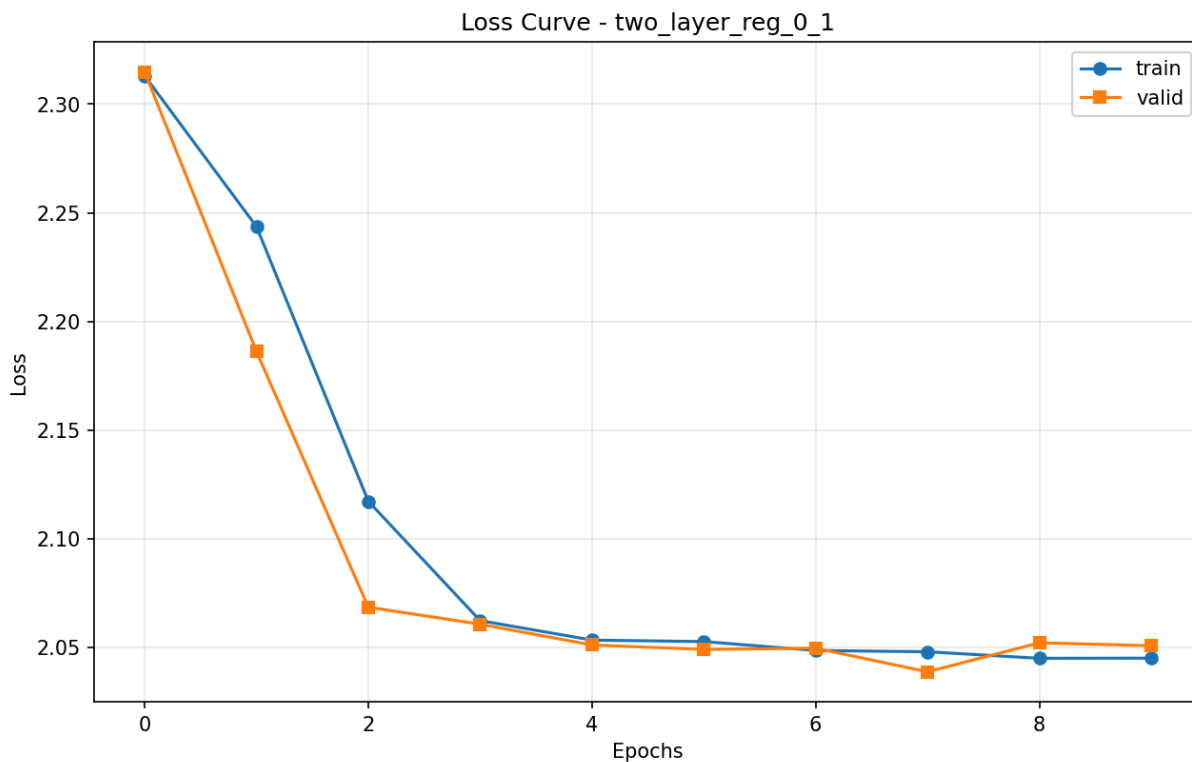
```
Starting training for 10 epochs...
Training: 100% 10/10 [01:10<00:00, 7.05s/it]
```

```
Training completed! Test Accuracy: 0.3549, Best Val Accuracy: 0.3577
Training log saved to: plots/two_layer_reg_0_1.log
Loss plot saved to: plots/two_layer_reg_0_1_loss.png
Accuracy plot saved to: plots/two_layer_reg_0_1_accuracy.png
```

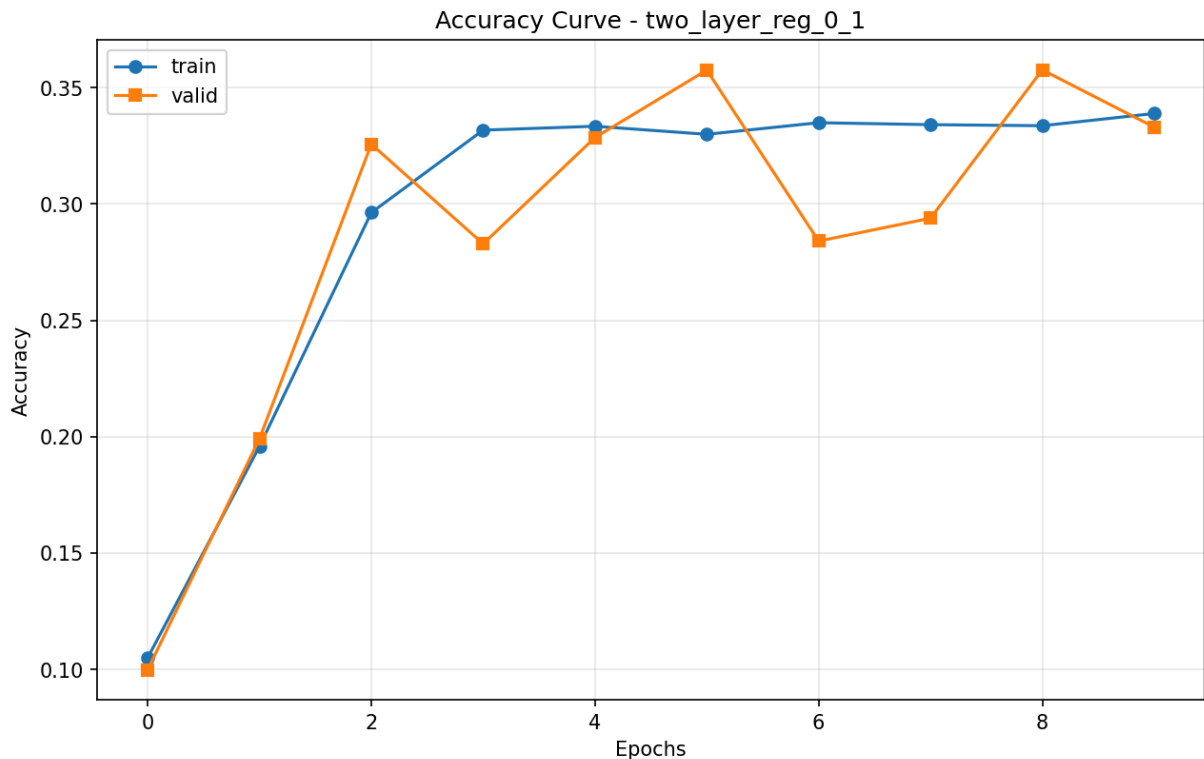
```
In [26]: display_training_plots("two_layer_reg_0_1")
```

```
Run: two_layer_reg_0_1
Test Accuracy: 0.3549
Best Validation Accuracy: 0.3577
Model: TwoLayerNet, LR: 0.1, Reg: 0.1
```

Loss Curve:



Accuracy Curve:



In [27]: `!python train_model.py --config configs/config_twolayer.yaml --run_name two_`

Starting training run: two_layer_reg_0_01

Configuration: {'batch_size': 64, 'learning_rate': 0.1, 'reg': 0.01, 'epochs': 10, 'momentum': 0.9, 'debug': True, 'type': 'TwoLayerNet', 'hidden_size': 128}

Loading training data...

Training data loaded with 60000 images

Loading testing data...

Testing data loaded with 10000 images

Using TwoLayerNet model with hidden_size=128

Using SGD optimizer with lr=0.1, reg=0.01

Starting training for 10 epochs...

Training: 100% 10/10 [01:05<00:00, 6.55s/it]

Training completed! Test Accuracy: 0.8923, Best Val Accuracy: 0.8955

Training log saved to: plots/two_layer_reg_0_01.log

Loss plot saved to: plots/two_layer_reg_0_01_loss.png

Accuracy plot saved to: plots/two_layer_reg_0_01_accuracy.png

In [28]: `display_training_plots("two_layer_reg_0_01")`

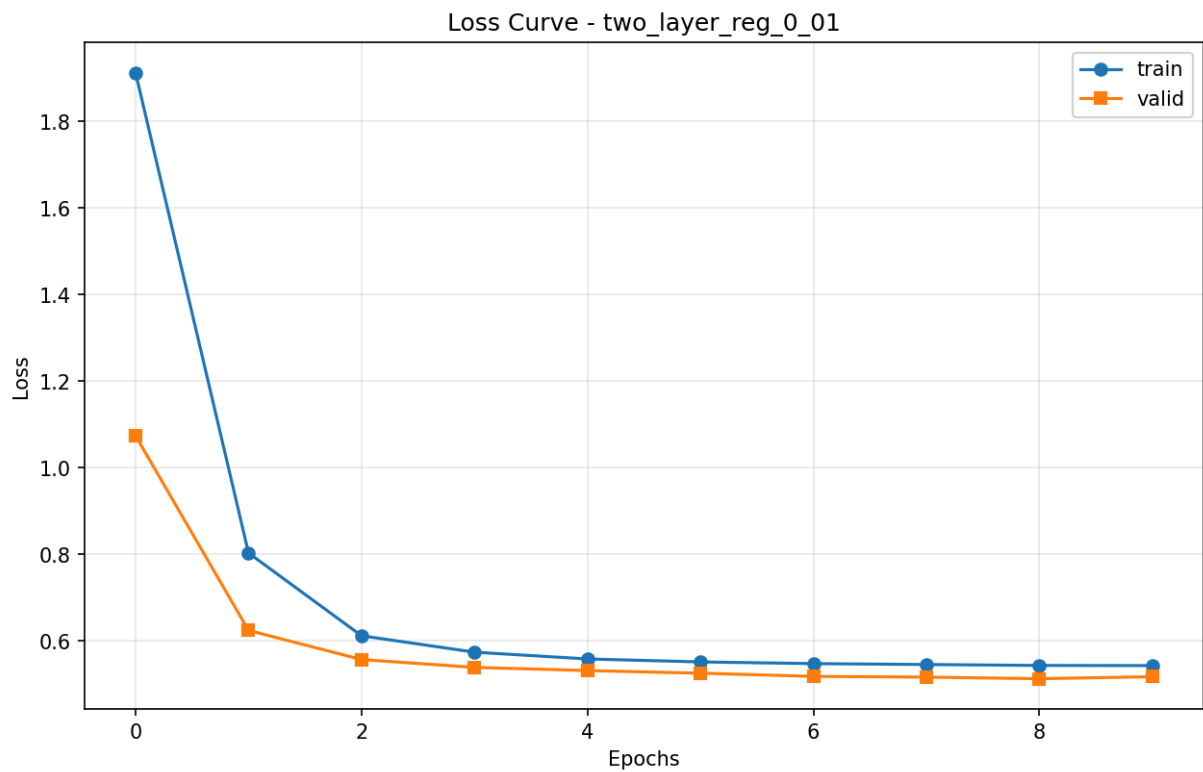
Run: two_layer_reg_0_01

Test Accuracy: 0.8923

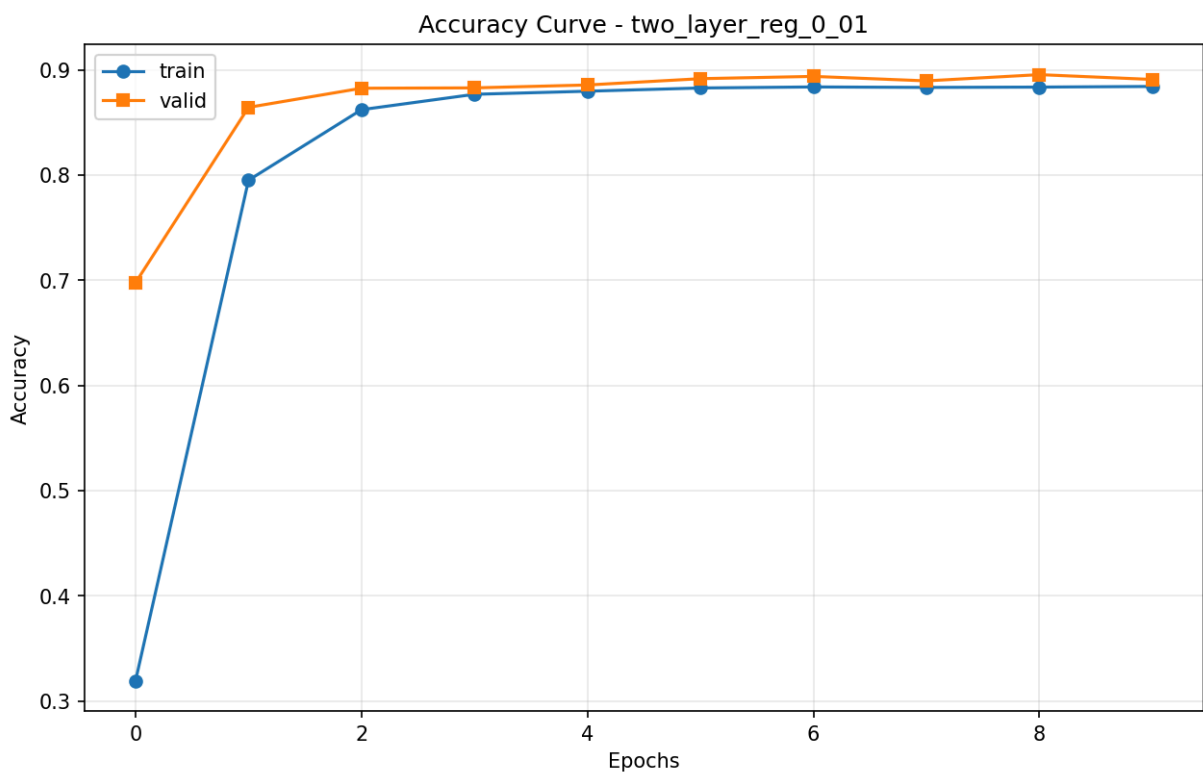
Best Validation Accuracy: 0.8955

Model: TwoLayerNet, LR: 0.1, Reg: 0.01

Loss Curve:



Accuracy Curve:



```
In [29]: !python train_model.py --config configs/config_twolayer.yaml --run_name two_
```

```
Starting training run: two_layer_reg_0_001
Configuration: {'batch_size': 64, 'learning_rate': 0.1, 'reg': 0.001, 'epochs': 10, 'momentum': 0.9, 'debug': True, 'type': 'TwoLayerNet', 'hidden_size': 128}
Loading training data...
Training data loaded with 60000 images
Loading testing data...
Testing data loaded with 10000 images
Using TwoLayerNet model with hidden_size=128
Using SGD optimizer with lr=0.1, reg=0.001
```

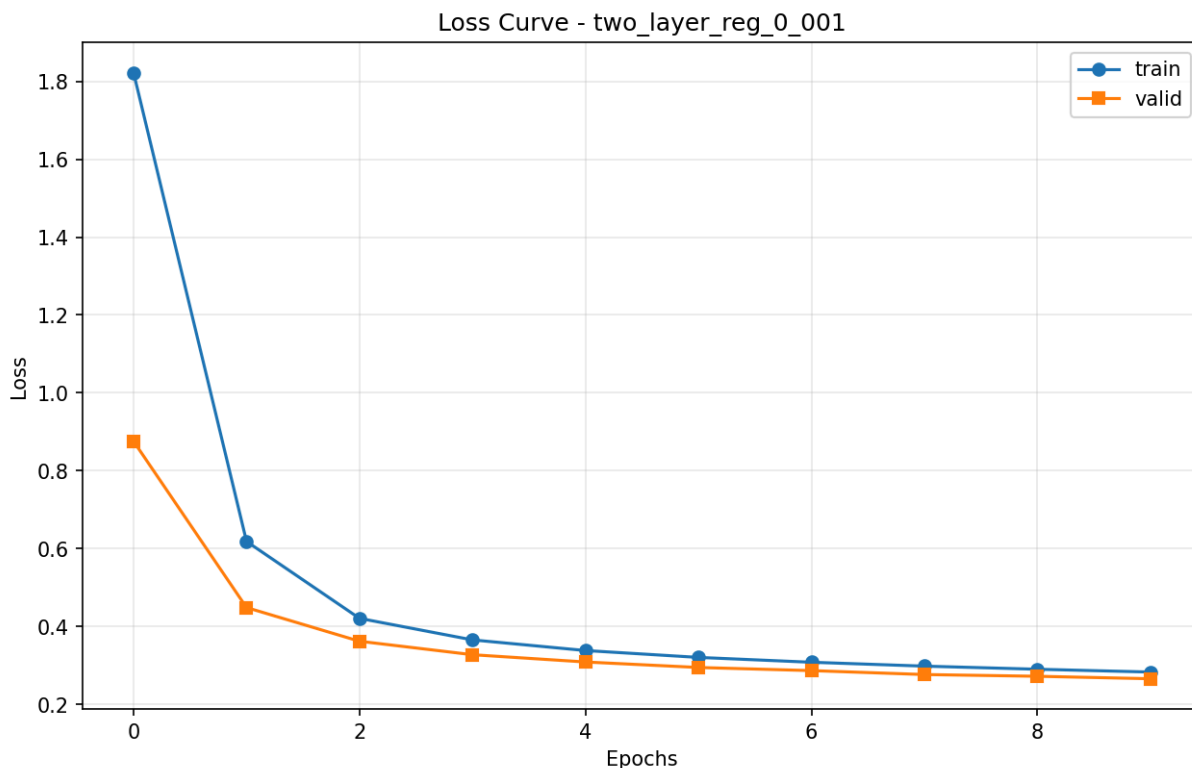
```
Starting training for 10 epochs...
Training: 100% 10/10 [01:08<00:00, 6.87s/it]
```

```
Training completed! Test Accuracy: 0.9254, Best Val Accuracy: 0.9264
Training log saved to: plots/two_layer_reg_0_001.log
Loss plot saved to: plots/two_layer_reg_0_001_loss.png
Accuracy plot saved to: plots/two_layer_reg_0_001_accuracy.png
```

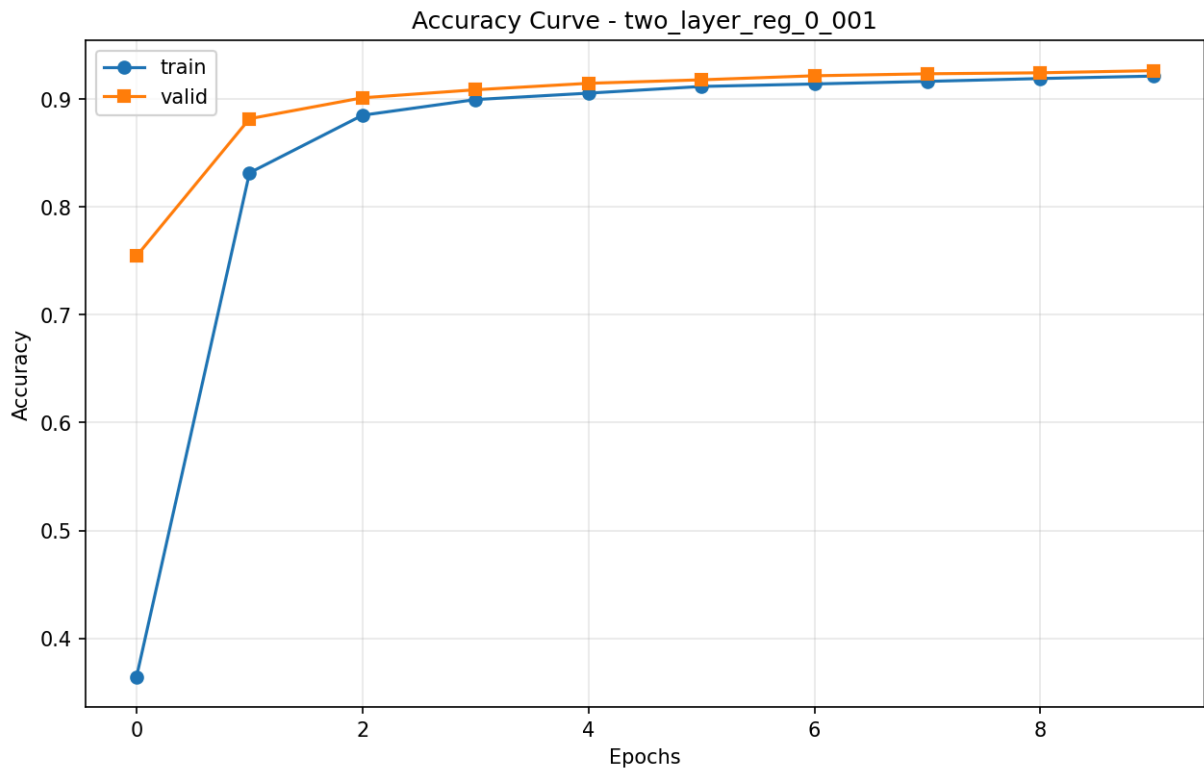
```
In [30]: display_training_plots("two_layer_reg_0_001")
```

```
Run: two_layer_reg_0_001
Test Accuracy: 0.9254
Best Validation Accuracy: 0.9264
Model: TwoLayerNet, LR: 0.1, Reg: 0.001
```

Loss Curve:



Accuracy Curve:



```
In [31]: !python train_model.py --config configs/config_twolayer.yaml --run_name two_
```

```
Starting training run: two_layer_reg_0_0001
Configuration: {'batch_size': 64, 'learning_rate': 0.1, 'reg': 0.0001, 'epochs': 10, 'momentum': 0.9, 'debug': True, 'type': 'TwoLayerNet', 'hidden_size': 128}
```

```
Loading training data...
```

```
Training data loaded with 60000 images
```

```
Loading testing data...
```

```
Testing data loaded with 10000 images
```

```
Using TwoLayerNet model with hidden_size=128
```

```
Using SGD optimizer with lr=0.1, reg=0.0001
```

```
Starting training for 10 epochs...
```

```
Training: 100% 10/10 [01:05<00:00, 6.55s/it]
```

```
Training completed! Test Accuracy: 0.9332, Best Val Accuracy: 0.9343
```

```
Training log saved to: plots/two_layer_reg_0_0001.log
```

```
Loss plot saved to: plots/two_layer_reg_0_0001_loss.png
```

```
Accuracy plot saved to: plots/two_layer_reg_0_0001_accuracy.png
```

```
In [32]: display_training_plots("two_layer_reg_0_0001")
```

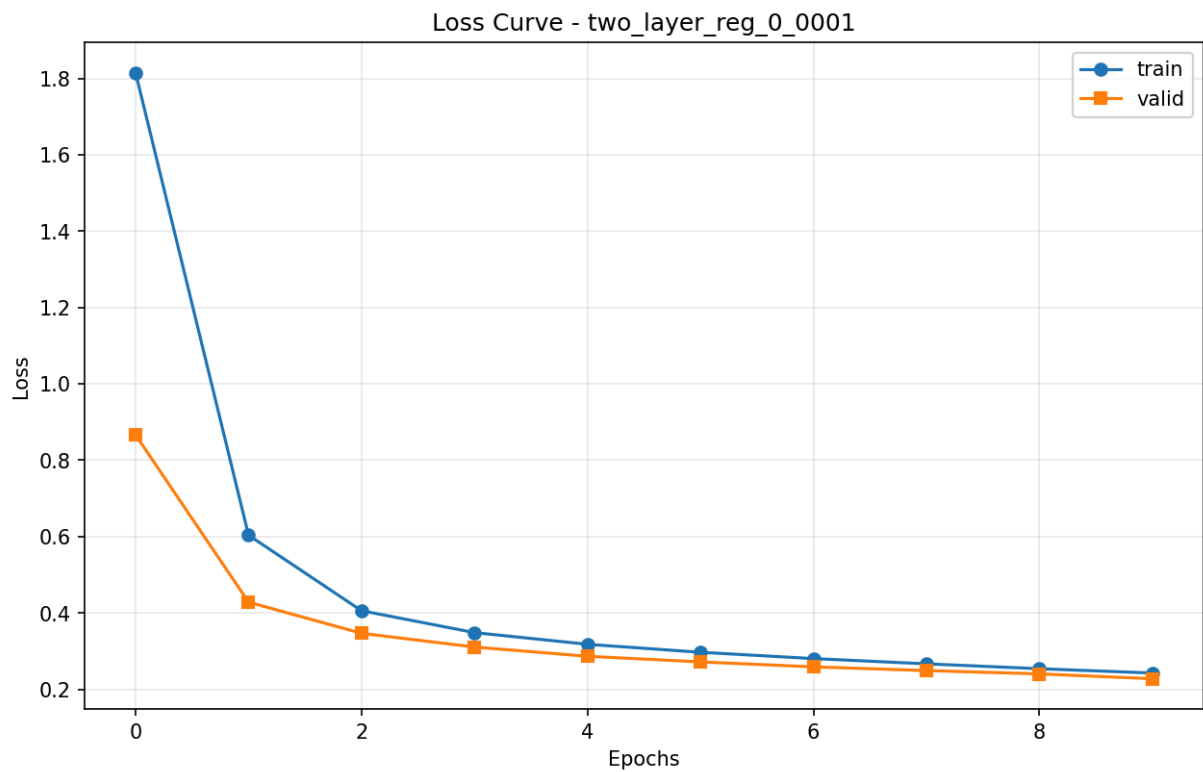
```
Run: two_layer_reg_0_0001
```

```
Test Accuracy: 0.9332
```

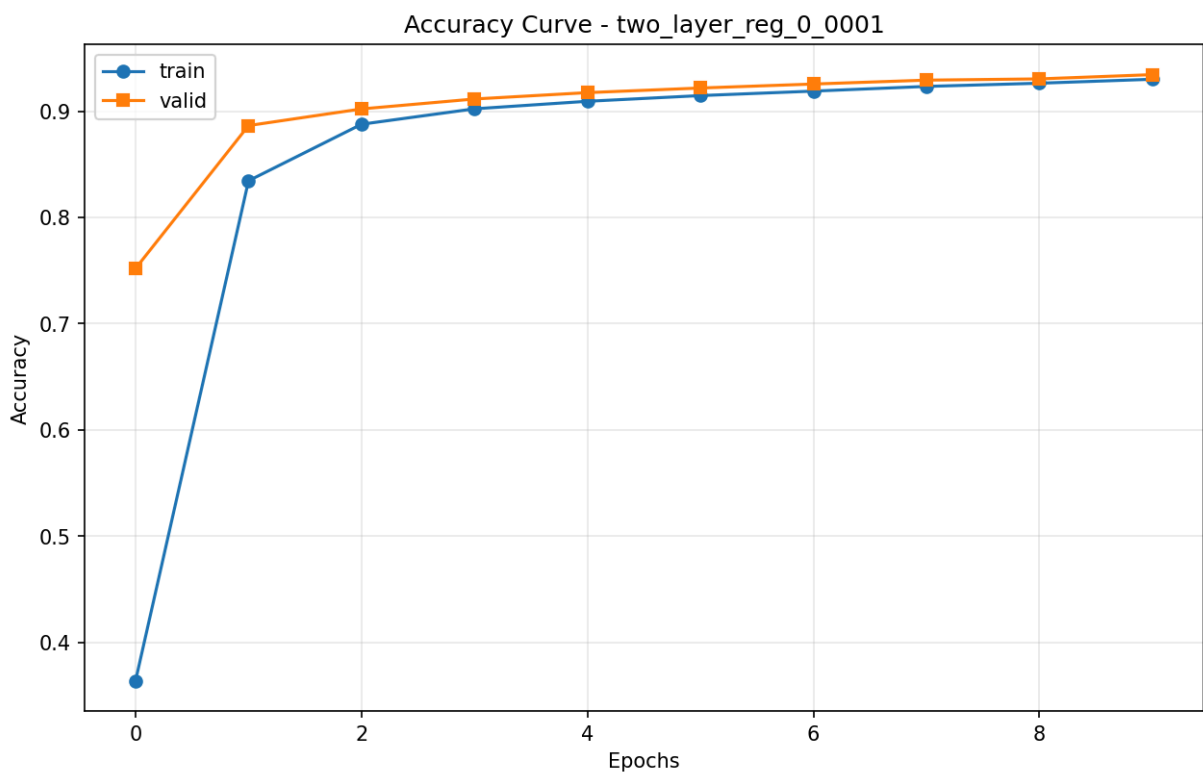
```
Best Validation Accuracy: 0.9343
```

```
Model: TwoLayerNet, LR: 0.1, Reg: 0.0001
```

```
-----
Loss Curve:
```



Accuracy Curve:



In [33]: `!python train_model.py --config configs/config_twolayer.yaml --run_name two_`

```
Starting training run: two_layer_reg_1
Configuration: {'batch_size': 64, 'learning_rate': 0.1, 'reg': 1.0, 'epochs': 10, 'momentum': 0.9, 'debug': True, 'type': 'TwoLayerNet', 'hidden_size': 128}
Loading training data...
Training data loaded with 60000 images
Loading testing data...
Testing data loaded with 10000 images
Using TwoLayerNet model with hidden_size=128
Using SGD optimizer with lr=0.1, reg=1.0
```

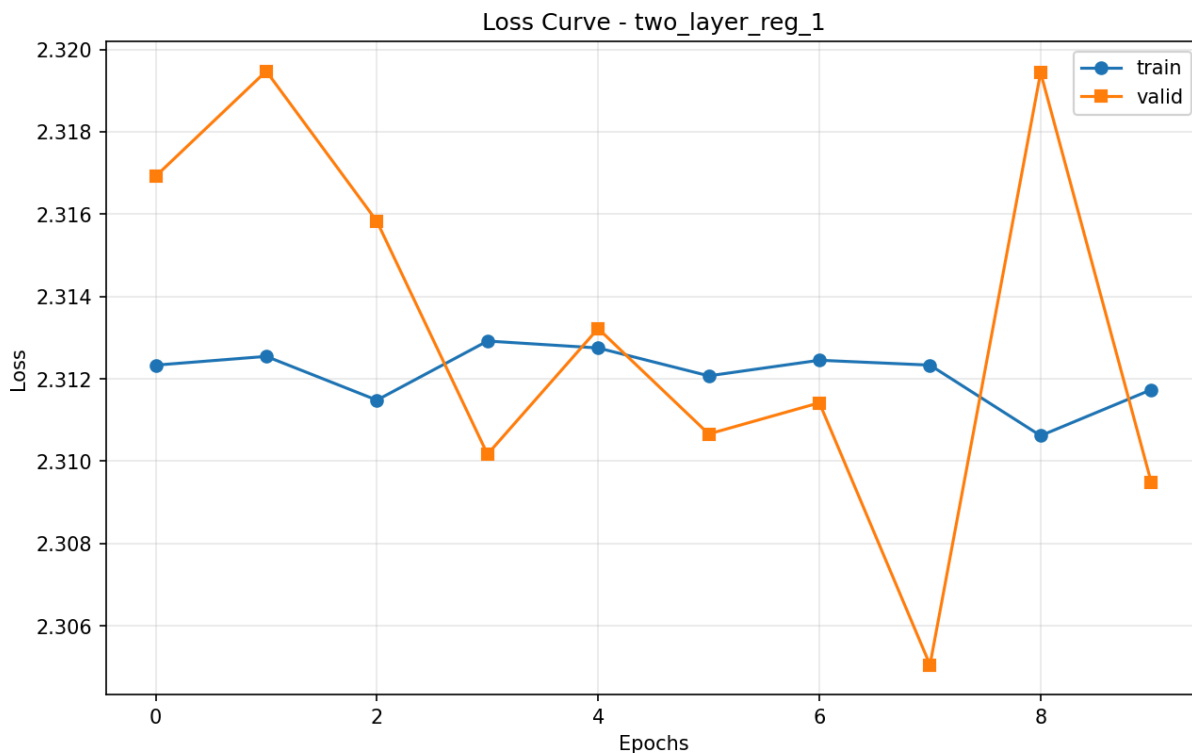
```
Starting training for 10 epochs...
Training: 100% 10/10 [01:08<00:00, 6.82s/it]
```

```
Training completed! Test Accuracy: 0.1028, Best Val Accuracy: 0.1081
Training log saved to: plots/two_layer_reg_1.log
Loss plot saved to: plots/two_layer_reg_1_loss.png
Accuracy plot saved to: plots/two_layer_reg_1_accuracy.png
```

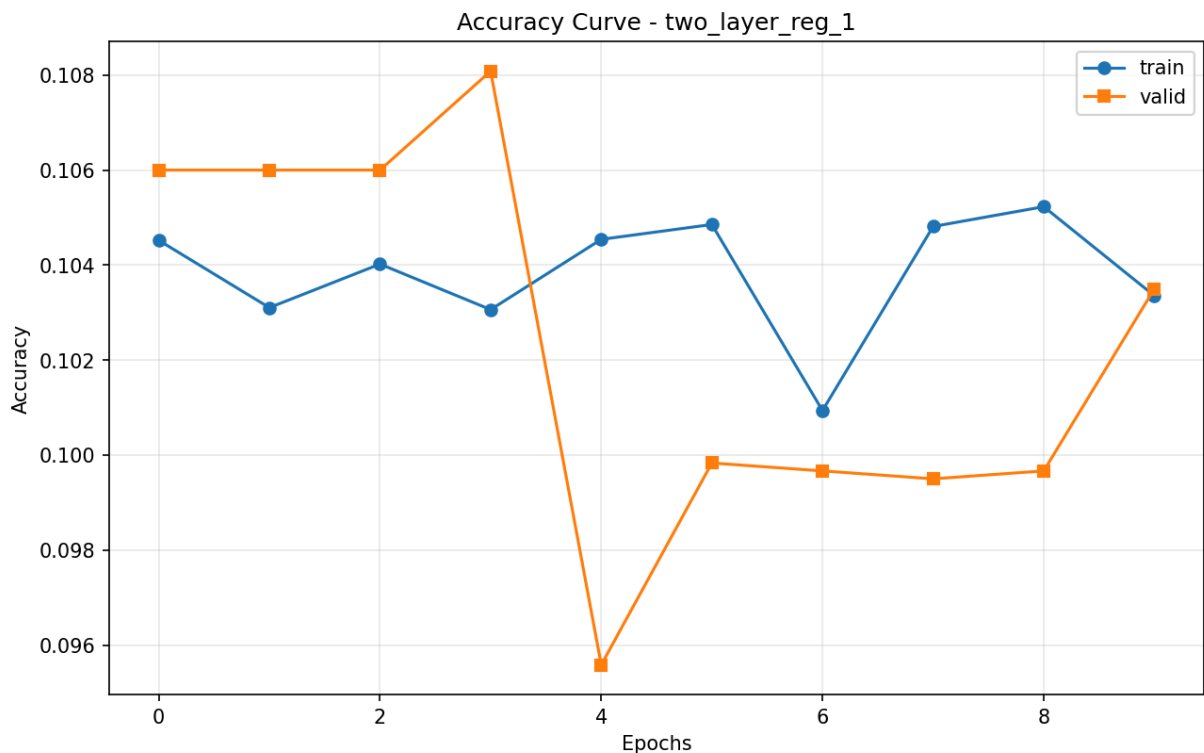
```
In [34]: display_training_plots("two_layer_reg_1")
```

```
Run: two_layer_reg_1
Test Accuracy: 0.1028
Best Validation Accuracy: 0.1081
Model: TwoLayerNet, LR: 0.1, Reg: 1.0
```

Loss Curve:



Accuracy Curve:



TODO: Observations [1 pt]

What differences do you observe between different regularization penalties? Why do you think this is the case?

Across the five experiments, what we observe is a clear inverse relationship between regularization coefficient and model accuracy. At the highest penalty of $\text{reg} = 1.0$, the network fails to learn and hovers not much better than random guessing at approximately 10.8% accuracy with erratic fluctuations. As regularization is reduced to $\text{reg} = 0.1$, the model still severely underfits at around 35.8% accuracy. However, when we decrease regularization further to $\text{reg} = 0.01$, 0.001 , and 0.0001 , accuracy climbs significantly to 89.6%, 92.6%, and reaches its peak at 93.4%, accompanied by smooth and stably monotonic loss and accuracy curves.

This behavior occurs because the optimization objective includes an L2 penalty term. When λ , the coefficient, is excessively large (≥ 0.1), this weight decay term dominates the cross-entropy gradient, aggressively forcing the weights toward a low magnitude (zero) and introducing high bias. This severely restricts the neural network's representational capacity and prevents it from fitting into the non-linear decision boundaries needed to distinguish MNIST digits. Conversely, small regularization values (10^{-3} and 10^{-4}) provide mild weight decay without stifling the

network's expressive power, allowing it to fit the dataset features and achieve strong generalization.

Hyperparameter Tuning

You are now free to tune any hyperparameters for better accuracy. Change `config_best.yaml` to find the best hyperparameters for your model.

In the previous sections, you should have achieved around 90% validation accuracy for some configurations. Try to tune the parameters to get at least (or close to) 95% accuracy.

When you are making changes to the regularization/learning rate values in the `.yaml` files, please do not use scientific notation in the `.yaml` files i.e. instead of writing `1e-1` please write `0.1`.

Plots [1 pt]

```
In [35]: !python train_model.py --config configs/config_best.yaml --run_name best_model
```

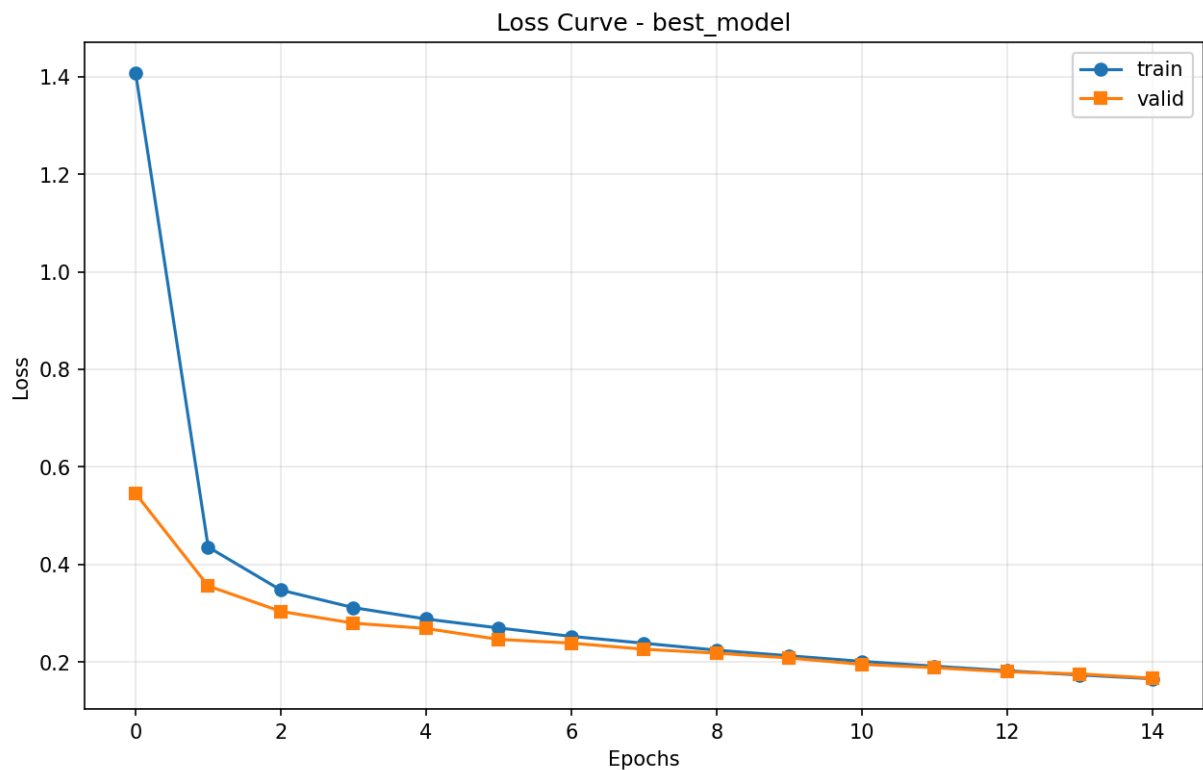
```
Starting training run: best_model
Configuration: {'batch_size': 64, 'learning_rate': 0.15, 'reg': 0.0001, 'epochs': 15, 'momentum': 0.9, 'debug': True, 'type': 'TwoLayerNet', 'hidden_size': 256}
Loading training data...
Training data loaded with 60000 images
Loading testing data...
Testing data loaded with 10000 images
Using TwoLayerNet model with hidden_size=256
Using SGD optimizer with lr=0.15, reg=0.0001

Starting training for 15 epochs...
Training: 100% 15/15 [02:27<00:00, 9.82s/it]

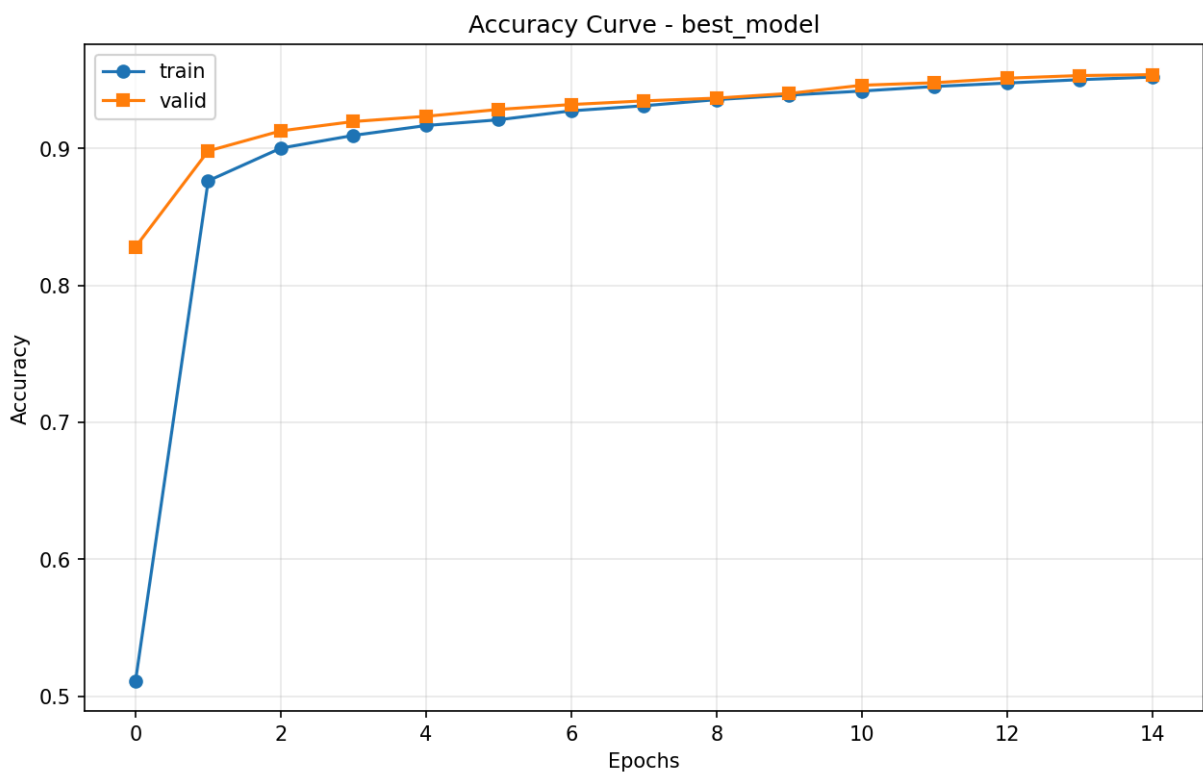
Training completed! Test Accuracy: 0.9511, Best Val Accuracy: 0.9539
Training log saved to: plots/best_model.log
Loss plot saved to: plots/best_model_loss.png
Accuracy plot saved to: plots/best_model_accuracy.png
```

```
In [36]: display_training_plots("best_model")
```

```
Run: best_model
Test Accuracy: 0.9511
Best Validation Accuracy: 0.9539
Model: TwoLayerNet, LR: 0.15, Reg: 0.0001
-----
Loss Curve:
```



Accuracy Curve:



TODO: Observations [1 pt]

How did you tune your hyperparameters? What did you try? What worked? What didn't?

I utilized the findings from the previous experiments, regarding the monotonicity of the loss and accuracy based on the trends in

the regularization coefficient, and the learning rate. Based on that, I decreased the regularisation coefficient gradually, all the way down to 0.0001 for this final plot, given the network's ability to fit the model properly without much requirement of regularization, even on the test set. I raised the learning rate also a bit to 0.15 to accelerate Gradient Descent. To expand the network's representation learning capabilities, I chose to use the TwoLayerNet rather than SoftmaxRegression, based on some of my observations given my parameters, and doubled the hidden layer size (number of neurons) to ensure more dense connectivity. This combination, along with 15 epochs of training, ensured I got past 95% on both validation and testing accuracy.

Submission

IMPORTANT: Before submitting, please clear all output cells besides the plots showing loss and accuracy curves.

Colab may take some time to update on the cloud file system, so if you find that running this cell does not generate the latest version of your notebook, you can wait a few minutes before trying again, or just download the notebook and run the command locally.

```
In [ ]: # on colab, you may need to install chromium to convert the notebook to pdf
!playwright install-deps chromium
!playwright install chromium
```

```
In [ ]: !python ./collect_submission.py
```

```
=== HW1 Submission Collection Script ===
```

```
Converting notebook to PDF...
```

```
Successfully converted main.ipynb to hw1_notebook_submission.pdf
```

```
Creating zip archive...
```

```
Created hw1_code_submission.zip
```

```
=====
```

```
=== Submission Collection Complete ===
```

```
Generated: hw1_notebook_submission.pdf
```

```
Generated: hw1_code_submission.zip
```

```
=====
```

```
Please submit these files to Gradescope:
```

- hw1_notebook_submission.pdf
- hw1_code_submission.zip

```
Congratulations on completing the assignment!
```

In []: